

8,9. Laboratory Session - Memory Managers

Mark as done

Physical Memory Management

- Physical memory is organized in frames - continuous regions of 4 KB of physical memory
- The OS is responsible for managing the allocation of frames
- The physical memory already mapped by the initial paging tables needs to be marked as utilized so as to not be remapped to other virtual addresses
 - i.e. the first 4 MB of physical memory [0x0, 0x400'000)

```
#ifndef TRUE
#define TRUE ( 1 == 1 )
#endif
```

```
#ifndef FALSE
#define FALSE ( 1 == 0 )
#endif
```

```
typedef BYTE BOOLEAN;
```

Add the following functionality

```
// allocates a new frame. Returns TRUE if the frame was successfully allocated, FALSE if there are no more free frames
```

```
BOOLEAN frame_alloc(QWORD *frame, DWORD frame_count);
```

```
// frees a previously allocated frame through frame_alloc

void frame_free(QWORD frame, DWORD frame_count);

// returns TRUE if all frames in the region are free or FALSE otherwise

BOOLEAN are_frames_free(QWORD start_frame, DWORD frame_count);
```

For the E820 physical map you can use [bochs_map.h](https://wiki.osdev.org/Bochs_Map_H)

References:

- [https://wiki.osdev.org/Detecting_Memory_\(x86\)](https://wiki.osdev.org/Detecting_Memory_(x86))
- <http://www.uruk.org/orig-grub/mem64mb.html>
- https://wiki.osdev.org/Page_Frame_Allocation

Virtual Memory Management

- Virtual memory is organized in pages - continuous regions of 4KB of virtual memory
- The OS is responsible for the management of virtual pages

Add the following functionality:

```
// maps page_count new pages. Returns TRUE if the page was successfully mapped, FALSE if there are no more
free frames or pages.

// If *page != NULL then the function will map the frame to *page, else the allocator will determine the
virtual address

// If frame == -1 then this function internally calls frame_alloc to obtain the frames to which the pages
will be mapped - it is not necessary for the physical frames to be continuous

// If frame != -1 then the pages will be mapped to [frame, frame+page_count)

BOOLEAN page_alloc( void **page, DWORD page_count, QWORD frame);

// unmaps the pages and if free_backing is TRUE then frees the backing physical frame as well
```

```
// virtual address must be invalidated from CPU TLB cache, you can use the __invlpg() intrinsic to achieve
this

void page_free( void * page, DWORD page_count, BOOLEAN free_backing) ;

// returns TRUE if the page is mapped

BOOLEAN is_page_mapped( void * page)
```

Heap

- Heaps allow a more efficient use of memory by offering a finer granularity when doing memory allocations
- Instead of allocating memory at page granularity the heap granularity will be at byte level
- For this each heap must maintain additional information about its allocations

Add the following functionality:

```
// Creates a new heap. This function internally calls page_alloc to allocate the pages in which the heap will
reside.

BOOLEAN heap_create( HEAP* heap, void* base, DWORD size)

// Allocate from the heap a buffer of Size bytes or returns NULL if the allocation cannot be done

void *heap_alloc( HEAP* heap, DWORD size) ;

// Free a heap entry previously allocated through heap_alloc

void heap_free( HEAP* heap, void* address) ;

// Destroys a heap. The pages allocated in the heap will be freed

void heap_destroy(HEAP* heap) ;
```

References:

- General advice before developing a heap https://wiki.osdev.org/Writing_a_memory_manager

- <https://wiki.osdev.org/User:Pancakes/SimpleHeapImplementation#Implementations> - 5 different heap implementations
- https://wiki.osdev.org/Memory_Allocation
- <http://www.osdever.net/tutorials/view/memory-management-2>

Suggested data structures - **NOT mandatory** and may be different depending on your heap implementation

```
typedef struct _LIST_ENTRY {
    struct _LIST_ENTRY *Flink;
    struct _LIST_ENTRY *Blink;
} LIST_ENTRY;

typedef struct _HEAP_HEADER{
    LIST_ENTRY    Entry;
    DWORD         Size;
}HEAP_HEADER;

typedef struct _HEAP
{
    void*        Base;
    DWORD        Size;
    LIST_ENTRY   FreeList;
}HEAP;
```

Assignment:

- Implement the functionality described for each type of allocator:
 - physical memory **frame_**
 - virtual memory **page_**
 - heap memory **heap_**
- Add tests for each function implemented. Add new commands to the console **test_run <num test>**, a command **test_list** which displays the names of the tests and a command **test_run_all** which runs all the tests.
 - test_run page
 - allocate a memory page using **page_alloc**
 - write data in the buffer
 - allocate a new memory page using **page_alloc** specifying the same frame used initially
 - check that the data in the pages are identical

- unmap the first page without freeing the backing physical frames
- verify that the data can still be accessed using the second allocation
 - test_run heap
- Create a heap using **heap_create**
- Allocate multiple heap entries using **heap_alloc**
- Write distinct data in each entry (you can use a counter)
- Validate the data in the entries and check if the values correspond
- Free the entries using **heap_free**
- Delete the heap using **heap_destroy**
- (OPTIONAL) Add support for lazy mapping
 - Add a new BOOLEAN parameter *is_lazy_mapped* to **page_alloc**
 - If TRUE the page won't be through the paging tables, but the OS will store this mapping and mark the virtual page as used
 - If FALSE behaviour doesn't change
 - When lazily mapped these accesses will generate page faults (#PF) so the OS needs to check if the virtual memory has been lazily mapped and if so map it through the paging tables and return from the exception
- (OPTIONAL) For correct physical memory management the OS is responsible for determining the memory map of the platform. The memory map, known also as the E820h map is obtained using the BIOS interrupt service BIOS INT15 with eax = 0xE820 - services can only be called in 16-bit real-mode. This is the only method to obtain the memory map in legacy boot. More details available on [osdev](http://www.osdev.org/osdev) and <http://www.uruk.org/orig-grub/mem64mb.html>

Deadline: 7.12 23:55

References:

- [__invlpg\(\)](#)
- https://wiki.osdev.org/Brendan%27s_Memory_Management_Guide
- <http://www.osdever.net/tutorials/view/memory-management-1>
- <http://www.osdever.net/tutorials/view/memory-management-2>

Last modified: Thursday, 20 November 2025, 11:12 AM